AMITY UNIVERSITY, MUMBAI

Amity School of Engineering and Technology

Department of Computer Science & Engineering

MINI PROJECT SYNOPSIS

(Academic Year 2025–26)

A Multi-Agent Web-Based System for AI-Driven Secure Cryptographic Key Generation with Entropy Validation and Security Analysis

Submitted By:

1. Khurram Rashid, A70405223016
2. Kinza Zahra, A70405223156
3. Prapti Patil, A70405223086
4. Shreya Deshpande, A70405223198

Under the Guidance of:

Name of Guide: Dr. Sarang Maruti Patil
Designation: _____

1. Title of the Project

A Multi-Agent Web-Based System for AI-Driven Secure Cryptographic Key Generation with Entropy Validation and Security Analysis

2. Abstract

Cryptographic key generation is a critical component of modern cybersecurity systems used in banking, cloud platforms, and secure communication [1]. The security of these systems depends on the randomness of generated keys. However, weak entropy sources and improper random number generation often produce predictable keys, creating serious vulnerabilities that can be exploited without breaking encryption algorithms [2], [4].

To address this issue, the proposed project introduces a **multi-agent web-based system** for AI-driven secure cryptographic key generation with entropy validation and security analysis. The system supports standard algorithms such as RSA, ECC, and AES. A KeyGeneratorAgent generates keys, while an AIEntropyAgent analyzes randomness using machine learning techniques. A MathModelAgent performs statistical tests, and a CryptoStrengthAgent evaluates resistance to brute-force attacks.

The system also includes features such as weak key detection, compliance checking, secure key storage, and key rotation policies. It will be developed as a web application using Python (FastAPI/Flask), React, and machine learning libraries like TensorFlow or PyTorch.

The expected outcome is a platform capable of generating highly secure and unpredictable cryptographic keys, improving security, reliability, and real-world applicability.

3. Problem Statement

Cryptographic key generation is essential for securing digital systems such as online banking, cloud services, and authentication mechanisms [1]. The strength of these systems depends on the randomness of generated keys. However, many systems still rely on weak entropy sources or poorly implemented random number generators, leading to predictable or low-entropy keys and serious security vulnerabilities [2], [3].

Real-world incidents highlight this issue. The Debian OpenSSL vulnerability produced predictable keys due to insufficient entropy, exposing millions of systems [7]. Additionally, weaknesses in cryptographic implementations and random number generation standards further demonstrate the risks of improper key generation practices [4].

Existing tools primarily focus on key generation but lack mechanisms for randomness validation, pattern detection, and security strength evaluation [13], [14]. They also fail to integrate entropy validation, statistical testing, and AI-based analysis into a unified system.

Therefore, there is a need for an intelligent system that not only generates cryptographic keys but also validates their randomness and evaluates their security. This project addresses this gap by proposing a multi-agent, AI-driven web-based platform for secure key generation and validation.

4. Objectives

Objective 1: To design and develop a web-based system for generating cryptographic keys using standard algorithms such as RSA, ECC, and AES with configurable parameters.

Objective 2: To implement entropy-based randomness validation using Shannon entropy and statistical methods to ensure the unpredictability of generated keys.

Objective 3: To integrate AI-based analysis for detecting patterns and weaknesses in generated keys using machine learning models.

Objective 4: To evaluate the security strength of keys by estimating resistance to brute-force and other potential attacks.

Objective 5: To provide key management features such as secure storage, key rotation policies, and a visualization dashboard for monitoring key security metrics.

5. Scope of the Project

The proposed project focuses on developing a **multi-agent web-based system** for secure cryptographic key generation, entropy validation, and security analysis. It supports key generation using standard algorithms (RSA, ECC, AES), evaluates randomness through entropy calculations and statistical tests, and applies AI-based techniques to detect patterns or weaknesses in generated keys.

The system also includes features such as key strength evaluation, compliance checking, secure storage, key rotation policies, and a visualization dashboard for monitoring security metrics. It is implemented as a web application, enabling users to generate and analyze keys for applications such as banking, API security, and data encryption.

Designed as a scalable prototype, the system ensures modular and extensible architecture through multi-agent integration. It can be further extended to enterprise-level deployment, with future enhancements including advanced cryptographic infrastructures, real-time systems, and support for emerging technologies like quantum-resistant cryptography.

6. Literature Survey / Existing System

Cryptographic key generation is widely used in modern systems; however, many approaches rely on traditional pseudo-random generators and lack proper entropy validation and security analysis [1], [2]. Incidents such as the Debian OpenSSL vulnerability demonstrate how low entropy can produce predictable keys and expose systems [7], while weaknesses in standard random number generation further highlight security risks [4].

Several real-world breaches emphasize poor key management practices. Cases like the Capital One breach and Uber (2016) incident show how misconfigurations and exposed credentials can compromise sensitive data [8], [15]. Similarly, AWS key exposure, GitHub token leaks, and phishing attacks reveal risks due to insecure storage and weak authentication [9], [11], [12]. The Heartbleed vulnerability and IoT-related issues further demonstrate how improper key generation and handling can lead to large-scale security failures [10], [13].

Existing systems mainly focus on key generation and lack integrated mechanisms for entropy validation, AI-based analysis, and security evaluation [13], [14]. Therefore, there is a need for an advanced system that combines entropy validation, security analysis, and intelligent techniques. The proposed system addresses these gaps through a multi-agent, AI-driven web-based platform for secure key generation and validation.

Table 1: Summary of Existing Systems and Their Limitations

Existing System / Incident	**Approach Used**	**Limitation / Drawback**	**Ref**
Debian OpenSSL vulnerability	Pseudo-random generator	Low entropy → predictable keys	[7]
Dual_EC_DRBG	Standard RNG algorithm	Potential backdoor, predictable output	[16]
Heartbleed vulnerability	OpenSSL cryptography	Private keys leaked from memory	[10]
Capital One data breach	Cloud key management	Misconfigured security, weak key control	[8]
Uber (2016 breach)	Credential-based access	Poor key storage and exposure	[15]
AWS Key Exposure Cases	API key usage	Keys leaked due to improper handling	[11]
GitHub Token Leaks	Access tokens	No secure storage / rotation	[12]
Google Docs phishing attack	Authentication system	Weak verification mechanisms	[9]
IoT Devices (Multiple cases)	Embedded key generation	Same or predictable keys across devices	[13]

Online Key Generators	Basic key generation	No entropy validation or security analysis	[13], [14]

7. Proposed System

The proposed system is a **multi-agent web-based platform** designed to generate, validate, and manage cryptographic keys using artificial intelligence and entropy-based analysis. Unlike existing systems that only focus on key generation, this system integrates multiple layers of validation and analysis to ensure that the generated keys are highly secure and resistant to real-world attacks.

The system consists of several specialized agents working together. Initially, the **KeyGeneratorAgent** generates cryptographic keys using standard algorithms such as RSA, ECC, and AES based on user-defined requirements. The generated keys are then passed to the **EntropyCalculationAgent**, which computes Shannon entropy to measure the randomness of the key. Following this, the **AIEntropyAgent** uses machine learning models to analyze patterns and detect any predictability in the generated sequences.

To further strengthen validation, the **MathModelAgent** performs statistical randomness tests such as frequency and autocorrelation analysis. The **CryptoStrengthAgent** then evaluates the key's resistance to brute-force and other attacks by estimating computational complexity. Additionally, a **PerformanceAgent** monitors system efficiency, including key generation time and resource usage.

The system also includes practical features such as secure key storage in an encrypted vault, key rotation policies based on usage (e.g., banking or API keys), compliance checking with standards like NIST and OWASP, and a user-friendly dashboard for visualization of entropy scores and security metrics.

This proposed system improves existing solutions by combining AI-based analysis, mathematical validation, and multi-agent architecture into a single platform. It ensures that keys are not only generated but also verified for randomness, strength, and security, making them significantly more difficult to predict or break compared to traditional methods.

8. Methodology

• Requirement Analysis

This phase analyzes the need for a secure cryptographic key generation system based on challenges such as weak entropy sources, predictable random number generation, and poor key management [2], [3]. Incidents like the Debian OpenSSL vulnerability highlight how insufficient randomness can produce predictable keys and compromise security [7]. Accordingly, the system's functional requirements include key generation using RSA, ECC, and AES, randomness evaluation using Shannon entropy [1] and statistical tests [4], and AI-based detection of patterns. It also supports secure storage, key rotation, compliance with NIST and OWASP standards [4], [5], and visualization of security metrics.

The non-functional requirements focus on security, performance, scalability, and usability. The system is designed to generate keys efficiently with low latency, support multiple users, and ensure data confidentiality. It follows a modular and extensible architecture for future enhancements and provides a simple user interface for better understanding of security metrics.

• Design (UML Diagrams)

The system architecture is designed using UML diagrams to represent different components and their interactions. A multi-agent architecture is used where agents such as KeyGeneratorAgent, EntropyCalculationAgent, AIEntropyAgent, MathModelAgent, and CryptoStrengthAgent work together. Data Flow Diagrams (DFD) are used to show how data moves through the system, and system architecture diagrams illustrate the interaction between frontend, backend, and database components.

The overall system architecture showing interaction between frontend, backend, and multi-agent modules is illustrated below in Fig.1.

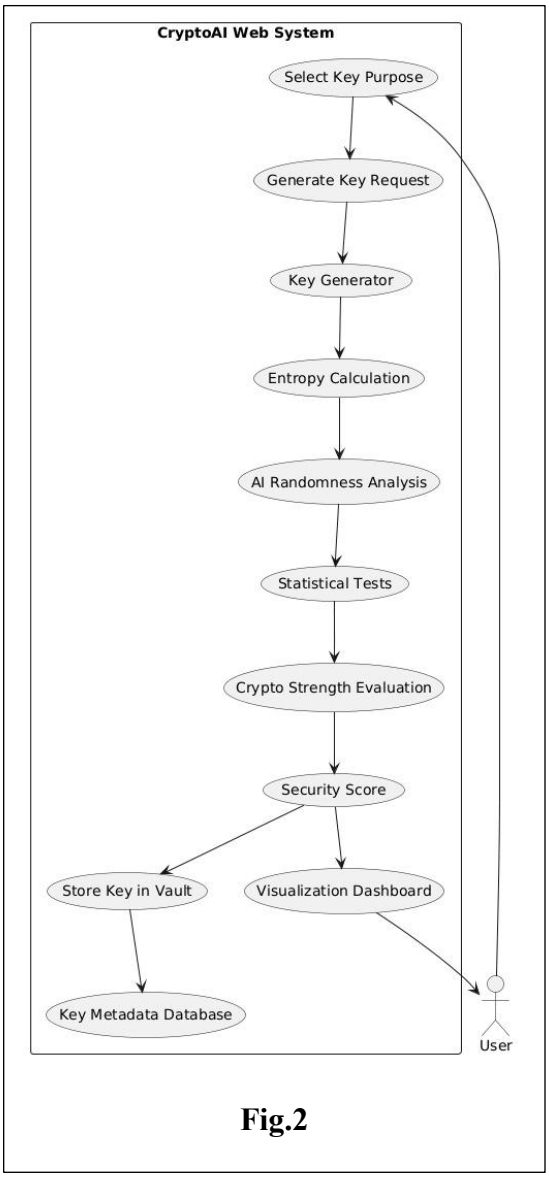

Fig.2

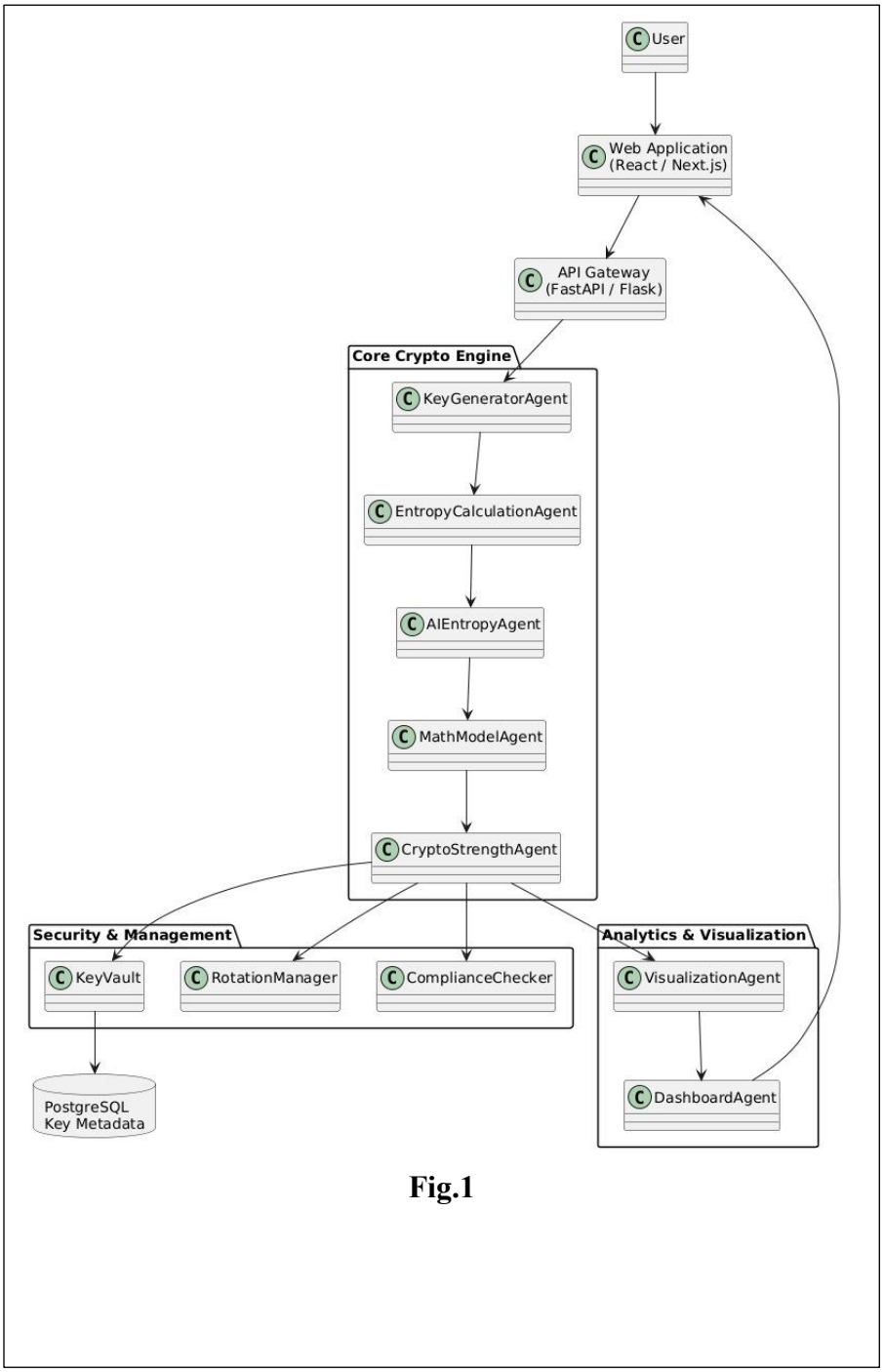

Fig.1

The data flow diagram in Fig.2 represents the step-by-step process of key generation and validation within the system.

The detailed multi-agent workflow in Fig.3 demonstrates how different agents collaborate to ensure secure key generation and validation.

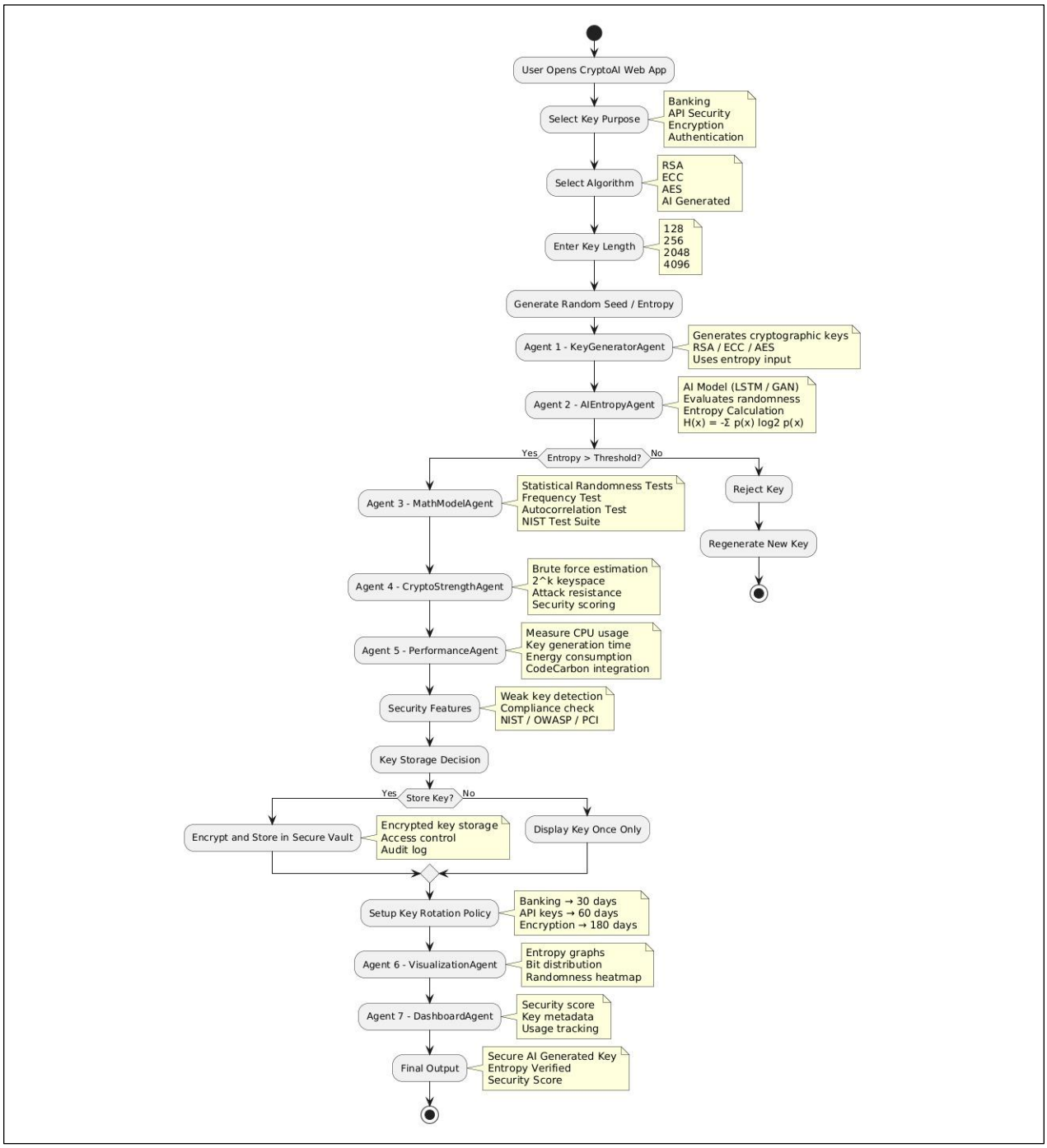

Fig.3

• Development

The system is developed as a web application using technologies such as React for frontend and Python (FastAPI/Flask) for backend. Cryptographic operations are implemented using standard libraries, while machine learning models are integrated for entropy and pattern analysis. Each agent is developed as a modular component to ensure scalability and maintainability.

• Testing

The system is tested using different scenarios to ensure correctness and security. Randomness of generated keys is validated using entropy calculations and statistical tests. Security testing includes checking resistance to brute-force attacks and detection of weak keys. Performance testing is conducted to evaluate system efficiency and response time.

9. Tools & Technologies

Programming Language:

Python (for backend development and AI/ML integration), JavaScript (for frontend development)

Database:

MongoDB / Firebase /PostgreSQL (for storing key metadata, user data, and key management information)

Tools:

• React / Next.js (Frontend development)
• FastAPI / Flask (Backend API development)
• PyTorch / TensorFlow (Machine Learning models for entropy and pattern analysis)
• Cryptography / PyCryptodome libraries (for RSA, ECC, AES key generation)
• Chart.js / Plotly (for visualization of entropy and security metrics)
• Postman (API testing)

• Git & GitHub (version control and collaboration)
• VS Code (development environment)

10. System Requirements

Hardware:

• Processor: Intel i5 or higher (or equivalent)
• RAM: Minimum 8 GB (16 GB recommended for ML models)
• Storage: Minimum 256 GB HDD/SSD
• Internet Connection: Required for web application access and deployment

Software:

• Operating System: Windows / Linux / macOS
• Web Browser: Google Chrome / Mozilla Firefox
• Programming Environment: Python (3.x), Node.js
• Frameworks: React / Next.js (Frontend), FastAPI / Flask (Backend)
• Machine Learning Libraries: TensorFlow / PyTorch, Scikit-learn
• Database: MongoDB / Firebase
• Development Tools: VS Code, Git, GitHub
• API Testing Tool: Postman

11. Expected Outcomes

The proposed system delivers a reliable web-based platform for secure cryptographic key generation and validation. It generates keys using standard algorithms (RSA, ECC, AES) and ensures high randomness through entropy evaluation and statistical testing. AI-based analysis is used to detect patterns or weaknesses, improving key security over traditional methods.

The system produces highly unpredictable keys resistant to brute-force attacks and provides a security score with detailed analysis. It also includes features such as secure storage, key rotation, compliance checking, and a visualization dashboard for monitoring security metrics. Overall, the project offers a scalable and extensible solution for enhancing cryptographic security in real-world applications.

12. Timeline / Work Plan (March 3rd week to May 1st week)

Phase	Activity	Duration
1	Requirement Analysis	1 Week (March 3rd Week)
2	Design (UML Diagrams & System Architecture)	1 Week (March 4th Week)
3	Development (Frontend, Backend, AI Integration)	4 Weeks (April 1st – April 4th Week)
4	Testing & Final Integration	1 Week (May 1st Week)

13. References

[1] H. Corrigan-Gibbs, S. Mu, D. Boneh, and B. Ford, "Ensuring High-Quality Randomness in Cryptographic Key Generation," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2013.

[2] M. De Bernardi et al., "Pseudo-Random Number Generation using Generative Adversarial Networks," *arXiv preprint arXiv:1810.00378*, 2018.

[3] M. Alshammari et al., "Effective Pseudo-Random Number Generator Using Deep Convolution GAN," *Information*, vol. 16, no. 1, 2025.

[4] National Institute of Standards and Technology (NIST), "Recommendation for Random Number Generation Using Deterministic Random Bit Generators," NIST SP 800-90A, 2015.

[5] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2021.

[6] OpenSSL Project, "OpenSSL: Cryptography and SSL/TLS Toolkit Documentation."

[7] Debian Security Advisory, "Predictable Random Number Generator Vulnerability," 2008.

[8] A. Greenberg, “The Capital One Hack,” *Wired*, 2019.

[9] Google Security Blog, “Protecting you against phishing,” 2017.

[10] Codenomicon, “The Heartbleed Bug (CVE-2014-0160),” 2014.

[11] Amazon Web Services (AWS), “Best Practices for Managing AWS Access Keys.”

[12] GitHub Docs, “Keeping Your Account and Data Secure.”

[13] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed., Pearson, 2017.

[14] B. Schneier, *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, 2nd ed., Wiley, 1996.

[15] Uber Technologies, “Uber Data Breach Incident,” 2016.

[16] D. Shumow and N. Ferguson, “On the Possibility of a Back Door in the NIST SP800-90 Dual EC PRNG,” *Crypto 2007 Rump Session*, 2007.